

GUÍA DE LABORATORIO N.º 06

Manejo de datos con Spring Boot, JPA, Hibernate y operaciones CRUD

Evolución de la API REST: de almacenamiento en memoria a persistencia real con MySQL

Curso	Desarrollo Web Integrado
Unidad	Unidad 2 - Back-end con bases de datos
Semana	6
Duración sugerida	3 horas
Modalidad	Laboratorio práctico guiado
Producto esperado	API REST CRUD de productos persistida en MySQL

Esta práctica inicia la Unidad 2. Se parte de la API de productos construida en la semana 5 y se reemplaza el almacenamiento temporal en memoria por una capa de persistencia basada en Spring Data JPA e Hibernate.

1. Logro de aprendizaje

Al finalizar el laboratorio, el estudiante implementa persistencia de datos en una API REST Spring Boot mediante Spring Data JPA, Hibernate y MySQL, ejecutando correctamente operaciones CRUD y manteniendo una arquitectura separada en Controller, Service, Repository y Entity.

2. ¿Qué cambia respecto de la semana 5?

Semana 5	Semana 6
Los productos viven en memoria.	Los productos se almacenan en MySQL.
Al reiniciar la aplicación se pierden los datos.	Los datos permanecen después de reiniciar.
Producto es una clase Java común.	Producto se convierte en una entidad JPA.
El Service administra una colección.	El Service delega la persistencia al Repository.
No existe capa de datos real.	Se incorpora ProductoRepository.

Idea central: No se reescribe toda la API. Se conserva la responsabilidad del Controller y del Service, y se sustituye el mecanismo de almacenamiento. Esta es una ventaja directa de trabajar por capas.

3. Fundamentos teóricos necesarios

3.1. ¿Qué es persistencia?

Persistir significa conservar la información más allá de la ejecución temporal del programa. Una variable o colección en memoria desaparece al cerrar la aplicación; una base de datos permite conservar y consultar esa información posteriormente.

3.2. ¿Qué es ORM?

ORM (Object-Relational Mapping) permite relacionar objetos Java con tablas relacionales. En lugar de transformar manualmente cada objeto en una sentencia SQL, se define un mapeo y el framework realiza gran parte de ese trabajo.

Objeto Java Producto
 ⇕ ORM
 Tabla productos en MySQL

3.3. Diferencia entre JPA, Hibernate y Spring Data JPA

Tecnología	Rol en el proyecto
JPA / Jakarta Persistence	Especificación: define anotaciones y contratos estándar de persistencia.
Hibernate	Implementación ORM que traduce operaciones con entidades a SQL.
Spring Data JPA	Abstracción de Spring que genera repositorios y reduce código repetitivo.
MySQL	Motor relacional donde se almacenan físicamente los datos.

Controller
 ↓
 Service
 ↓
 Spring Data JPA Repository
 ↓
 JPA / Hibernate

↓
JDBC
↓
MySQL

4. Requisitos del laboratorio

- Java 25.
- Visual Studio Code con soporte Java/Spring.
- Maven o Maven Wrapper.
- MySQL Server activo.
- MySQL Workbench o cliente equivalente.
- Postman o Thunder Client.
- Proyecto de la semana 5 o un nuevo proyecto Spring Boot.

5. Preparar el proyecto

Si se reutiliza el proyecto de la semana 5, agregar dependencias de persistencia. Si se crea un proyecto nuevo en Spring Initializr, seleccionar: Spring Web, Spring Data JPA, MySQL Driver y Spring Boot Starter Test. Para ejecutar la prueba específica del Repository con `@DataJpaTest` en Spring Boot 4.x, añadir además `spring-boot-starter-data-jpa-test` con alcance `test`.

Buenas prácticas: No es necesario declarar manualmente la versión de cada dependencia administrada por Spring Boot. El BOM de Spring Boot mantiene versiones compatibles entre sí.

6. Verificar pom.xml

El archivo `pom.xml` debe contener al menos estas dependencias relevantes:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

<!-- Spring Boot 4.x: soporte específico para pruebas JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
```

7. Crear la base de datos MySQL

Abrir MySQL Workbench y ejecutar:

```
CREATE DATABASE productos_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE productos_db;
```

En esta etapa no es necesario crear manualmente la tabla productos. Hibernate podrá generarla a partir de la entidad durante el laboratorio.

8. Configurar application.properties

```
spring.application.name=productos-api

spring.datasource.url=jdbc:mysql://localhost:3306/productos_db?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=TU_PASSWORD

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false
```

Propiedad	Función
spring.datasource.url	Cadena JDBC para conectarse a MySQL.
username / password	Credenciales de conexión.
ddl-auto=update	Actualiza el esquema según las entidades durante el laboratorio.
show-sql=true	Muestra el SQL generado por Hibernate.
format_sql=true	Formatea el SQL para facilitar su lectura.
open-in-view=false	Evita mantener abierta la sesión de persistencia durante la capa web; favorece una separación más clara de responsabilidades.

Seguridad: Para un laboratorio local puede colocarse la contraseña en application.properties. En un entorno real deben utilizarse variables de entorno o gestores de secretos y nunca versionar credenciales.

9. Convertir Producto en una entidad JPA

Reemplazar el modelo de la semana 5 por una entidad. Observe que se utilizan anotaciones del paquete jakarta.persistence.

```
package com.utp.productosapi.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "productos")
```

```

public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String nombre;

    @Column(nullable = false)
    private double precio;

    @Column(nullable = false)
    private int stock;

    public Producto() {
    }

    public Producto(Long id, String nombre, double precio, int stock) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }
}

```

9.1. Explicación de las anotaciones

Anotación	Significado
@Entity	Indica que la clase será administrada por JPA.
@Table	Define la tabla asociada.

Anotación	Significado
@Id	Marca la clave primaria.
@GeneratedValue	Define cómo se generará el identificador.
GenerationType.IDENTITY	Utiliza el autoincremento del motor MySQL.
@Column	Permite configurar restricciones y características de la columna.

10. Crear el Repository

Crear el paquete repository y el archivo ProductoRepository.java:

```
package com.utp.productosapi.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.utp.productosapi.model.Producto;

public interface ProductoRepository extends JpaRepository<Producto, Long> {
}
```

No se implementa manualmente la interfaz. Spring Data JPA crea el objeto en tiempo de ejecución e incluye operaciones como save(), findAll(), findById(), existsById() y deleteById().

11. Refactorizar ProductoService

Eliminar el ConcurrentHashMap y AtomicLong utilizados en la semana 5. El servicio ahora recibe ProductoRepository mediante inyección por constructor.

```
package com.utp.productosapi.service;

import java.util.List;
import java.util.Optional;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.utp.productosapi.model.Producto;
import com.utp.productosapi.repository.ProductoRepository;

@Service
public class ProductoService {

    private final ProductoRepository repository;

    public ProductoService(ProductoRepository repository) {
        this.repository = repository;
    }

    @Transactional(readOnly = true)
    public List<Producto> listar() {
        return repository.findAll();
    }

    @Transactional(readOnly = true)
    public Optional<Producto> buscarPorId(Long id) {
        return repository.findById(id);
    }

    @Transactional
    public Producto crear(Producto producto) {
        return repository.save(producto);
    }
}
```

```

        validar(producto);
        producto.setId(null);
        return repository.save(producto);
    }

    @Transactional
    public Optional<Producto> actualizar(Long id, Producto datos) {
        validar(datos);
        return repository.findById(id).map(existente -> {
            existente.setNombre(datos.getNombre());
            existente.setPrecio(datos.getPrecio());
            existente.setStock(datos.getStock());
            return repository.save(existente);
        });
    }

    @Transactional
    public Optional<Producto> actualizarPrecio(Long id, double precio) {
        if (precio <= 0) {
            throw new IllegalArgumentException("El precio debe ser mayor que cero");
        }
        return repository.findById(id).map(existente -> {
            existente.setPrecio(precio);
            return repository.save(existente);
        });
    }

    @Transactional
    public boolean eliminar(Long id) {
        if (!repository.existsById(id)) {
            return false;
        }
        repository.deleteById(id);
        return true;
    }

    private void validar(Producto producto) {
        if (producto.getNombre() == null || producto.getNombre().isBlank()) {
            throw new IllegalArgumentException("El nombre es obligatorio");
        }
        if (producto.getPrecio() <= 0) {
            throw new IllegalArgumentException("El precio debe ser mayor que cero");
        }
        if (producto.getStock() < 0) {
            throw new IllegalArgumentException("El stock no puede ser negativo");
        }
    }
}

```

Sobre @Transactional: Aunque JpaRepository ya ejecuta operaciones transaccionales, colocar la frontera transaccional en el Service hace explícita la unidad de trabajo y prepara al estudiante para la semana 7, donde se profundizará en transacciones.

12. Mantener el Controller REST

El Controller de la semana 5 puede mantenerse prácticamente igual porque depende de ProductoService y no conoce cómo se almacenan los datos. Esto demuestra desacoplamiento entre la capa web y la capa de persistencia.

```

package com.utp.productosapi.controller;

import java.util.List;

```

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.utp.productosapi.model.Producto;
import com.utp.productosapi.service.ProductoService;

@RestController
@RequestMapping("/api/productos")
public class ProductoController {

    private final ProductoService service;

    public ProductoController(ProductoService service) {
        this.service = service;
    }

    @GetMapping
    public List<Producto> listar() {
        return service.listar();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Producto> buscar(@PathVariable Long id) {
        return service.buscarPorId(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Producto> crear(@RequestBody Producto producto) {
        Producto creado = service.crear(producto);
        return ResponseEntity.status(HttpStatus.CREATED).body(creado);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Producto> actualizar(
        @PathVariable Long id,
        @RequestBody Producto producto) {
        return service.actualizar(id, producto)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PatchMapping("/{id}/precio")
    public ResponseEntity<Producto> actualizarPrecio(
        @PathVariable Long id,
        @RequestParam double valor) {
        return service.actualizarPrecio(id, valor)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        return service.eliminar(id)
            ? ResponseEntity.noContent().build()
            : ResponseEntity.notFound().build();
    }
}

```

```
}
}
```

13. Ejecutar la aplicación

Verificar primero que MySQL esté iniciado. Luego, desde la terminal del proyecto:

```
mvn spring-boot:run
```

Durante el arranque, observar la consola. Hibernate debe establecer la conexión y crear o actualizar la tabla productos. Si `spring.jpa.show-sql=true`, las sentencias SQL serán visibles.

14. Verificar la tabla en MySQL

```
USE productos_db;
SHOW TABLES;
DESCRIBE productos;
SELECT * FROM productos;
```

La tabla debe contener columnas equivalentes a id, nombre, precio y stock.

15. Ejecutar las operaciones CRUD desde Postman

15.1. CREATE - crear producto

```
POST http://localhost:8080/api/productos
Content-Type: application/json

{
  "nombre": "Laptop Lenovo",
  "precio": 3500,
  "stock": 10
}
```

Resultado esperado: 201 Created y un JSON con id generado por MySQL.

15.2. READ - listar productos

```
GET http://localhost:8080/api/productos
```

Resultado esperado: 200 OK y una lista JSON.

15.3. READ - obtener por id

```
GET http://localhost:8080/api/productos/1
```

Resultado: 200 si existe; 404 si el id no está registrado.

15.4. UPDATE - actualizar producto completo

```
PUT http://localhost:8080/api/productos/1
Content-Type: application/json

{
  "nombre": "Laptop Lenovo Pro",
  "precio": 3900,
  "stock": 8
}
```

15.5. PATCH - actualizar solo el precio

```
PATCH http://localhost:8080/api/productos/1/precio?valor=4200
```

15.6. DELETE - eliminar producto

```
DELETE http://localhost:8080/api/productos/1
```

Resultado esperado: 204 No Content.

16. Comprobar que realmente existe persistencia

1. Crear dos productos desde Postman.
2. Consultar `SELECT * FROM productos;` en MySQL Workbench.
3. Detener la aplicación Spring Boot.
4. Volver a iniciar la aplicación.
5. Ejecutar `GET /api/productos`.
6. Confirmar que los productos siguen existiendo.

Conclusión experimental: Si los datos permanecen después de reiniciar Spring Boot, ya no dependen de la memoria del proceso: están persistidos en MySQL.

17. ¿Qué SQL genera Hibernate?

Observar la consola mientras se ejecutan las operaciones. El estudiante debería identificar sentencias equivalentes a:

```
insert into productos (...)
select ... from productos
update productos set ... where id=?
delete from productos where id=?
```

El objetivo no es ignorar SQL, sino comprender que JPA/Hibernate abstrae gran parte del código repetitivo de JDBC y genera SQL de acuerdo con las operaciones realizadas sobre las entidades.

18. Prueba de integración del Repository

Para comprobar el comportamiento de la capa de persistencia se puede usar `@DataJpaTest`. La prueba crea un producto y verifica que pueda recuperarse.

```
package com.utp.productosapi.repository;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.data.jpa.test.autoconfigure.DataJpaTest;

import com.utp.productosapi.model.Producto;

@DataJpaTest
class ProductoRepositoryTest {

    @Autowired
    private ProductoRepository repository;

    @Test
```

```

void debeGuardarYRecuperarProducto() {
    Producto producto = new Producto(null, "Monitor", 900.0, 5);

    Producto guardado = repository.save(producto);

    assertThat(guardado.getId()).isNotNull();
    assertThat(repository.findById(guardado.getId())).isPresent();
}
}

```

Atención: Con Spring Boot 4.x, `@DataJpaTest` pertenece al paquete `org.springframework.boot.data.jpa.test.autoconfigure` y el soporte se distribuye en el módulo `spring-boot-data-jpa-test`. Si VS Code no resuelve la anotación, verificar que la dependencia `spring-boot-starter-data-jpa-test` esté incluida con alcance `test`.

19. Errores frecuentes y diagnóstico

Síntoma	Causa probable	Qué revisar
Access denied for user	Credenciales MySQL incorrectas	username/password en <code>application.properties</code>
Unknown database	La base no existe o el nombre no coincide	CREATE DATABASE y URL JDBC
Communications link failure	MySQL detenido o puerto incorrecto	Servicio MySQL y puerto 3306
Table does not exist	Hibernate no creó la tabla	<code>@Entity</code> y <code>ddl-auto</code>
Not a managed type	Entidad fuera del escaneo o falta <code>@Entity</code>	Paquetes y anotación
404 en <code>/api/productos</code>	Ruta incorrecta o Controller no detectado	<code>@RestController</code> y <code>@RequestMapping</code>
500 al guardar	Dato inválido o error SQL	Consola de Spring y payload JSON

20. Buenas prácticas que deben quedar claras

- Controller atiende HTTP; no debe acceder directamente a MySQL.
- Service contiene reglas y coordina operaciones.
- Repository encapsula acceso a datos.
- Entity representa la estructura persistente.
- Las dependencias se inyectan por constructor.
- Las credenciales no deben versionarse en proyectos reales.
- `ddl-auto=update` es útil para desarrollo, pero no sustituye una estrategia de migraciones en producción.
- No se debe confundir Spring Data JPA con Hibernate: cumplen roles diferentes.

21. Reto práctico

Agregar el campo `categoria` y hacer que Hibernate actualice el esquema. Luego registrar productos de distintas categorías y comprobar el nuevo campo en MySQL. No crear todavía una consulta JPQL; ese tema corresponde a la semana 7.

```

@Column(nullable = false, length = 80)
private String categoria;

```

21. Preguntas de reflexión

1. ¿Cuál es la diferencia entre JPA e Hibernate?
2. ¿Qué ventaja ofrece Spring Data JPA frente a implementar acceso JDBC manual?
3. ¿Por qué ProductoRepository es una interfaz y aun así puede inyectarse como objeto?
4. ¿Qué función cumple @Entity?
5. ¿Qué ocurre con el identificador cuando se utiliza GenerationType.IDENTITY?
6. ¿Por qué el Controller de la semana 5 necesitó pocos cambios al introducir la base de datos?
7. ¿Qué riesgo existe al usar ddl-auto=update en producción?

22. Cierre

En esta semana la API evoluciona desde una solución temporal en memoria hacia una aplicación con persistencia real. El estudiante ya puede recorrer el flujo completo: petición HTTP → Controller → Service → Repository → JPA/Hibernate → MySQL. En la semana 7 se ampliará esta capa de datos mediante consultas JPQL y manejo de transacciones.